

ConferenceHub – Système de Gestion de Conférences

Architecture Micro-services avec Spring Cloud

Contexte

ConferenceHub est une plateforme dédiée à la gestion d'événements académiques et commerciaux. Elle permet d'organiser des conférences animées par des intervenants experts appelés **Keynotes**, de recueillir des avis du public sous forme de reviews, et de suivre en temps réel la popularité et le score de chaque conférence.

L'objectif est de proposer une solution moderne, scalable et sécurisée, capable de gérer efficacement :

- La gestion des intervenants (Keynotes) et de leurs profils
 - La planification et le suivi des conférences
 - La collecte et l'analyse des reviews du public
 - La communication asynchrone entre les services via un broker de messages
 - La supervision en temps réel de l'infrastructure et des micro-services
-

Mission du développeur

En tant que développeur, vous serez en charge de concevoir et développer une application basée sur une **architecture micro-services** avec **Spring Cloud**, en suivant les bonnes pratiques : séparation des responsabilités, tolérance aux pannes, documentation des API, sécurisation via Keycloak, observabilité complète et communication asynchrone via Kafka.

L'application devra :

- Exposer des API RESTful documentées avec OpenAPI/Swagger
 - Assurer la résilience des appels inter-services via Resilience4J
 - Publier et consommer des événements métier via Apache Kafka
 - Être surveillée en temps réel via Prometheus + Grafana + Zipkin
 - Être entièrement conteneurisée et déployable via Docker Compose
-

Fonctionnalités Principales

Module 1 : Gestion des Keynotes

- Ajouter un intervenant (Keynote)
- Modifier les informations d'un intervenant existant
- Supprimer un intervenant uniquement s'il n'est associé à aucune conférence active
- Consulter la liste complète des intervenants avec pagination
- Rechercher un intervenant par nom, prénom ou fonction
- Consulter le profil complet d'un intervenant avec ses conférences associées
- À chaque création d'un Keynote, publier un événement **keynote-created** sur Kafka

Module 2 : Gestion des Conférences

Conférences

- Créer une conférence et lui associer un ou plusieurs Keynotes
- Modifier les informations d'une conférence existante
- Supprimer une conférence si elle ne possède aucune review
- Consulter la liste complète des conférences avec pagination et filtres (type, date, score)
- Rechercher une conférence par titre ou type
- Calculer et mettre à jour automatiquement le score d'une conférence à partir de ses reviews
- À chaque création de conférence, publier un événement `conference-created` sur Kafka
- À chaque changement de statut, publier un événement `conference-status-changed` sur Kafka

Reviews

- Soumettre une review pour une conférence (texte + note 1 à 5)
- Modifier une review existante
- Supprimer une review
- Consulter toutes les reviews d'une conférence
- Recalculer automatiquement le score de la conférence après chaque opération sur une review
- À chaque soumission de review, publier un événement `review-submitted` sur Kafka

Inscriptions

- Inscrire un participant à une conférence
- Annuler une inscription
- Consulter la liste des inscrits d'une conférence
- Mettre à jour automatiquement le compteur `nombreInscrits` sur la conférence
- À chaque inscription, publier un événement `participant-registered` sur Kafka

Module 3 : Notification-Service

Le **notification-service** est un micro-service autonome dont le rôle est de **réagir aux événements métier** publiés sur Kafka par les autres services et d'effectuer les actions de notification correspondantes, sans jamais être appelé directement par les autres services.

Pourquoi ce service ?

Dans une architecture micro-services, il est mauvaise pratique que `conference-service` ou `keynote-service` s'occupent d'envoyer des emails ou de gérer l'historique des notifications. Cela crée du couplage et alourdit la logique métier. Le `notification-service` **sépare complètement cette responsabilité** : il écoute, il réagit, il notifie.

Ce que fait ce service concrètement

Événement reçu (Kafka)	Action déclenchée
------------------------	-------------------

Événement reçu (Kafka)	Action déclenchée
KEYNOTE_CREATED	Envoyer un email de bienvenue au nouveau Keynote
CONFERENCE_CREATED	Notifier les managers qu'une nouvelle conférence est planifiée
CONFERENCE_STATUS_CHANGED → EN_COURS	Envoyer un email de rappel à tous les inscrits de la conférence
CONFERENCE_STATUS_CHANGED → ANNULEE	Envoyer un email d'annulation à tous les inscrits
PARTICIPANT_REGISTERED	Envoyer un email de confirmation d'inscription au participant
REVIEW_SUBMITTED	Notifier le Keynote concerné qu'il a reçu un nouvel avis

API REST exposée par ce service

- GET /notifications → historique complet avec pagination
- GET /notifications?email=xxx → notifications d'un destinataire
- GET /notifications?conferenceId=xxx → notifications liées à une conférence
- GET /notifications?statut=ECHOUÉE → notifications en échec à relancer
- POST /notifications/retry/{id} → relancer manuellement une notification échouée

Règles de Gestion Globales

Keynotes

- Un Keynote peut intervenir dans plusieurs conférences
- La suppression d'un Keynote est impossible s'il est lié à au moins une conférence
- L'email d'un Keynote doit être unique dans le système

Conférences

- Une conférence peut avoir plusieurs Keynotes (relation ManyToMany)
- Une conférence passe par plusieurs statuts : PLANIFIEE → EN_COURS → TERMINEE | ANNULEE
- Le score est recalculé automatiquement à chaque ajout, modification ou suppression d'une review
- La suppression n'est possible que si la conférence ne possède aucune review
- Le nombre d'inscrits est géré via l'entité Inscription
- Une conférence a obligatoirement un titre, un type, une date et une durée

Reviews

- Une review appartient à une seule conférence
- La note est comprise entre 1 et 5 (validation stricte)
- La date est enregistrée automatiquement à la création

Inscriptions

- Un participant ne peut s'inscrire qu'une seule fois à une conférence donnée
- L'annulation d'une inscription décrémente le compteur `nombreInscrits`

Notifications

- Chaque notification est persistée en base de données avec son statut
- En cas d'échec d'envoi, la notification passe au statut `ECHOUÉE` et peut être relancée manuellement
- Une notification est toujours liée à un événement source identifiable (type + référence)

Gestion des Rôles

- `ROLE_ADMIN` : accès complet (CRUD sur tout + consultation historique notifications)
- `ROLE_MANAGER` : peut créer et modifier des keynotes et conférences, gérer les reviews
- `ROLE_USER` : lecture + soumission de reviews + inscription à une conférence

Modélisation des Entités

Keynote-Service

Keynote

Champ	Type	Contraintes	Description
id	Long	PK, auto	Identifiant unique
nom	String	NotBlank	Nom de l'intervenant
prenom	String	NotBlank	Prénom de l'intervenant
email	String	Unique, Email	Adresse email unique
fonction	String	NotBlank	Titre professionnel

Conference-Service

Conference

Champ	Type	Contraintes	Description
id	Long	PK, auto	Identifiant unique
titre	String	NotBlank	Titre de la conférence
type	Enum	NotNull	ACADEMIQUE, COMMERCIALE
statut	Enum	NotNull, défaut PLANIFIEE	PLANIFIEE, EN_COURS, TERMINEE, ANNULEE
date	LocalDate	NotNull, future	Date de la conférence
duree	Integer	Min(1)	Durée en minutes

Champ	Type	Contraintes	Description
nombreInscrits	Integer	Min(0), défaut 0	Nombre de participants inscrits
score	Double	calculé automatiquement	Moyenne des notes des reviews
keynoteIds	List<Long>	—	IDs des Keynotes (OpenFeign)
reviews	List<Review>	OneToMany, cascade	Reviews associées
inscriptions	List<Inscription>	OneToMany, cascade	Inscriptions associées
createdAt	LocalDateTime	auto	Date de création
updatedAt	LocalDateTime	auto	Date de dernière modification

Review

Champ	Type	Contraintes	Description
id	Long	PK, auto	Identifiant unique
date	LocalDate	auto à la création	Date de soumission
texte	String	NotBlank, max 1000	Contenu textuel de la review
stars	Integer	Min(1), Max(5)	Note de 1 à 5
conference	Conference	ManyToOne, NotNull	Conférence associée
auteurEmail	String	Email	Email de l'auteur (JWT claim)

Inscription

Champ	Type	Contraintes	Description
id	Long	PK, auto	Identifiant unique
conference	Conference	ManyToOne, NotNull	Conférence associée
participantEmail	String	Email, NotBlank	Email du participant inscrit
participantNom	String	NotBlank	Nom du participant
dateInscription	LocalDate	auto à la création	Date d'inscription
statut	Enum	défaut CONFIRMEE	CONFIRMEE, ANNULEE

ConferenceKeynote (table de jointure)

Champ	Type	Description
conferenceId	Long	FK vers Conference
keynoteId	Long	ID du Keynote (stocké localement en référence)

Notification-Service

Notification

Champ	Type	Contraintes	Description
id	Long	PK, auto	Identifiant unique
destinataire	String	Email, NotBlank	Email du destinataire
sujet	String	NotBlank	Sujet de l'email envoyé
contenu	String	NotBlank	Corps du message
typeEvenement	Enum	NotNull	KEYNOTE_CREATED, CONFERENCE_CREATED, STATUS_CHANGED, REVIEW_SUBMITTED, PARTICIPANT_REGISTERED
referenceld	Long	—	ID de l'entité source (conferenceld, keynoteld...)
statut	Enum	défaut EN_ATTENTE	EN_ATTENTE, ENVOYEE, ECHOUÉE
dateEnvoi	LocalDateTime	auto	Date et heure d'envoi effectif
tentatives	Integer	défaut 0	Nombre de tentatives d'envoi
erreurMessage	String	nullable	Message d'erreur en cas d'échec

Tolérance aux pannes – Resilience4J

Pourquoi Resilience4J ?

Dans notre architecture, **conference-service** appelle **keynote-service** de manière synchrone via OpenFeign pour récupérer les détails des Keynotes d'une conférence. Cette dépendance directe crée un risque : si keynote-service devient indisponible ou lent, conference-service peut à son tour se bloquer, puis tomber, provoquant une panne en cascade qui affecte toute l'application. Resilience4J est la bibliothèque de tolérance aux pannes qui protège contre ce scénario.

Pattern 1 : Circuit Breaker (Disjoncteur)

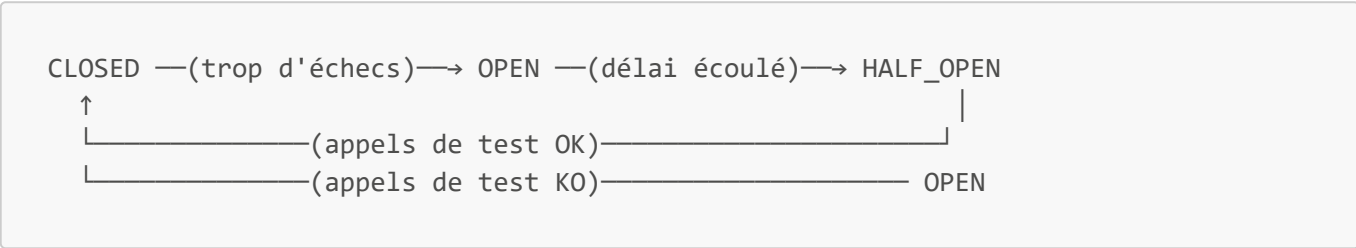
C'est le pattern central de Resilience4J. Il fonctionne exactement comme un disjoncteur électrique : il surveille les appels sortants et coupe la communication quand trop d'échecs sont détectés, protégeant ainsi les deux services.

Le circuit passe par trois états :

État CLOSED (circuit fermé — fonctionnement normal) Les appels vers keynote-service passent normalement. Resilience4J surveille en silence le taux d'échec sur une fenêtre glissante de N appels. Tant que le taux d'échec reste en dessous du seuil configuré (par exemple 50%), tout fonctionne normalement.

État OPEN (circuit ouvert — protection active) Dès que le taux d'échec dépasse le seuil, le circuit s'ouvre. Les appels vers keynote-service sont immédiatement **bloqués sans même être envoyés**. Resilience4J retourne directement une réponse de secours appelée **fallback** — par exemple la conférence est retournée sans les détails des Keynotes, avec un message indiquant que le service est temporairement indisponible. keynote-service n'est plus sollicité et peut récupérer tranquillement.

État HALF_OPEN (demi-ouvert — test de reprise) Après un délai d'attente configuré (par exemple 10 secondes), le circuit passe en demi-ouvert. Resilience4J laisse passer un nombre limité d'appels de test. Si ces appels réussissent, le circuit se referme et tout reprend normalement. Si les appels échouent encore, le circuit se rouvre pour un nouveau délai d'attente.



Pattern 2 : Retry (Réessai automatique)

Avant de considérer un appel comme définitivement échoué, Resilience4J peut le **réessayer automatiquement** plusieurs fois avec un délai croissant entre chaque tentative. Cela permet d'absorber les erreurs réseau transitoires qui se résolvent en quelques millisecondes sans déclencher le circuit breaker.

Par exemple : un premier appel échoue à cause d'une micro-coupure réseau → Resilience4J attend 500ms et réessaie → le deuxième appel réussit → l'utilisateur ne voit rien. Sans Retry, cet appel aurait directement échoué et compté comme un échec vers le circuit breaker.

Pattern 3 : TimeLimiter (Timeout)

Un service qui répond très lentement est aussi dangereux qu'un service qui ne répond pas du tout, car il monopolise des threads en attente. Le TimeLimiter coupe automatiquement tout appel qui dépasse un délai maximum configuré (par exemple 3 secondes) et déclenche le fallback. Sans ce mécanisme, un keynote-service dégradé qui répond en 30 secondes bloquerait tous les threads de conference-service.

Résumé des comportements dans ce projet

Situation	Sans Resilience4J	Avec Resilience4J
keynote-service est complètement KO	conference-service plante en cascade	Fallback retourné, conférence affichée sans détails keynote
keynote-service répond très lentement	Les threads de conference-service se bloquent	Appel coupé après 3s, fallback déclenché immédiatement
Erreur réseau brève et transitoire	Échec immédiat remonté à l'utilisateur	3 tentatives automatiques avant de déclencher le fallback

Situation	Sans Resilience4J	Avec Resilience4J
Trop d'échecs accumulés	Chaque appel attend et échoue un par un	Circuit ouvert, réponses immédiates, keynote-service protégé

Visibilité dans le monitoring

Les états du circuit breaker sont exposés via `/actuator/health` et `/actuator/prometheus`, ce qui permet à **Grafana** d'afficher en temps réel l'état de chaque circuit (CLOSED / OPEN / HALF_OPEN), le taux d'échec courant, le nombre d'appels bloqués et le nombre de retries effectués.

 Communication Asynchrone – Apache Kafka

Pourquoi Kafka ?

La communication synchrone via OpenFeign convient pour les requêtes qui nécessitent une réponse immédiate. En revanche, pour les événements métier qui doivent déclencher des traitements dans d'autres services sans couplage fort, on utilise **Apache Kafka** comme broker de messages.

Topics Kafka

Topic	Producteur	Consommateur(s)	Description
keynote-events	keynote-service	notification-service	Création d'un nouveau Keynote
conference-events	conference-service	notification-service	Création et changements de statut
review-events	conference-service	notification-service	Soumission de reviews
inscription-events	conference-service	notification-service	Inscriptions et annulations

Événements Kafka (Event Payloads)

KeynoteCreatedEvent

```
{
  "eventType": "KEYNOTE_CREATED",
  "timestamp": "2024-06-01T10:00:00",
  "keynoteId": 1,
  "nom": "Dupont",
  "prenom": "Jean",
  "email": "jean.dupont@email.com",
  "fonction": "Professeur"
}
```

ConferenceCreatedEvent


```
{
  "eventType": "CONFERENCE_CREATED",
  "timestamp": "2024-06-01T10:05:00",
  "conferenceId": 10,
  "titre": "IA & Société",
  "type": "ACADEMIQUE",
  "date": "2024-09-15",
  "statut": "PLANIFIEE"
}
```

ConferenceStatusChangedEvent

```
{
  "eventType": "CONFERENCE_STATUS_CHANGED",
  "timestamp": "2024-09-15T09:00:00",
  "conferenceId": 10,
  "ancienStatut": "PLANIFIEE",
  "nouveauStatut": "EN_COURS"
}
```

ReviewSubmittedEvent

```
{
  "eventType": "REVIEW_SUBMITTED",
  "timestamp": "2024-09-16T14:30:00",
  "reviewId": 5,
  "conferenceId": 10,
  "stars": 4,
  "auteurEmail": "user@email.com"
}
```

ParticipantRegisteredEvent

```
{
  "eventType": "PARTICIPANT_REGISTERED",
  "timestamp": "2024-08-20T11:00:00",
  "inscriptionId": 3,
  "conferenceId": 10,
  "participantEmail": "alice@email.com",
  "participantNom": "Alice Martin"
}
```

Schéma de communication

```

keynote-service
|— (OpenFeign sync) —————→ conference-service
|— (Kafka producer) —→ keynote-events —→ notification-service
|
conference-service
|— (Kafka producer) —→ conference-events —→ notification-service
|— (Kafka producer) —→ review-events —→ notification-service
|— (Kafka producer) —→ inscription-events —→ notification-service
|
notification-service
|— (Kafka consumer) ← tous les topics ci-dessus
|— (REST API)
    → GET /notifications
    → GET /notifications?email=xxx
    → GET /notifications?conferenceId=xxx
    → GET /notifications?statut=ECHOUÉE
    → POST /notifications/retry/{id}

```

Monitoring – Prometheus, Grafana & Zipkin

Pourquoi un stack de monitoring ?

Dans une architecture micro-services, chaque service est indépendant. Il est indispensable de centraliser les métriques, les logs et les traces distribuées pour détecter rapidement les anomalies de performance, les erreurs et les goulots d'étranglement.

Outils

Outil	Rôle	Port
Prometheus	Scraping et stockage des métriques exposées par chaque micro-service	9090
Grafana	Visualisation des métriques sous forme de dashboards interactifs	3000
Zipkin	Tracing distribué des requêtes inter-services (latence, erreurs)	9411

Prometheus

Chaque micro-service expose un endpoint `/actuator/prometheus` grâce à `spring-boot-starter-actuator` et `micrometer-registry-prometheus`. Prometheus scrape automatiquement ces endpoints selon l'intervalle configuré.

Grafana – Métriques visualisées

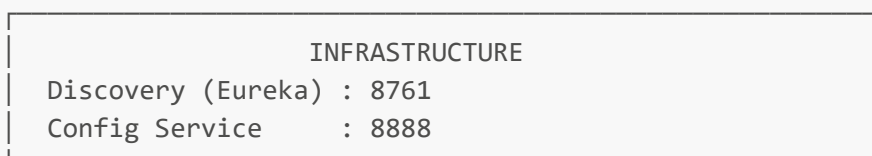
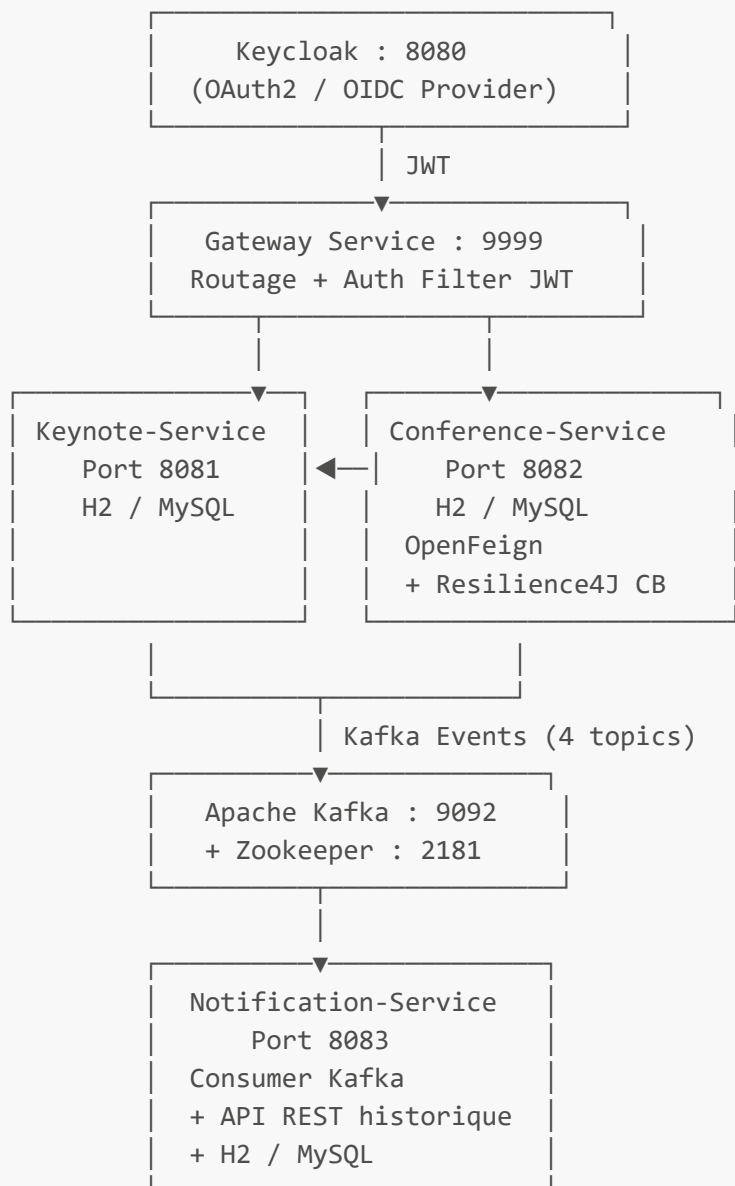
- Taux de requêtes HTTP par service (req/s)
- Temps de réponse moyen et percentile 95
- Nombre d'erreurs 4xx / 5xx
- Utilisation mémoire et CPU de chaque JVM
- Statut des circuit breakers Resilience4J (OPEN / CLOSED / HALF_OPEN)
- Taux d'échec et nombre de retries par instance de circuit breaker

- Lag des consumers Kafka par topic
- Nombre de notifications envoyées / échouées (métriques custom notification-service)

Zipkin – Tracing Distribué

Zipkin permet de tracer le chemin complet d'une requête à travers tous les micro-services impliqués. Par exemple, une requête de consultation de conférence passe par : Gateway → Conference-Service → Keynote-Service (Feign). Zipkin enregistre la latence de chaque segment et permet d'identifier quel service est responsable d'un ralentissement.

Architecture Technique Complète



MONITORING STACK

Prometheus : 9090 (scrape /actuator/prometheus)
 Grafana : 3000 (dashboards JVM + HTTP + Kafka
 + circuit breakers)
 Zipkin : 9411 (tracing distribué)

Structure du Projet Maven

```
conferences-app/
├── config-service/           ← Port 8888
├── discovery-service/       ← Port 8761
├── gateway-service/         ← Port 9999
├── keynote-service/         ← Port 8081
├── conference-service/      ← Port 8082
└── notification-service/    ← Port 8083 (consumer Kafka + API REST)
```

Déploiement Docker Compose

Service	Image de base	Port exposé
config-service	eclipse-temurin:21	8888
discovery-service	eclipse-temurin:21	8761
gateway-service	eclipse-temurin:21	9999
keynote-service	eclipse-temurin:21	8081
conference-service	eclipse-temurin:21	8082
notification-service	eclipse-temurin:21	8083
keycloak	quay.io/keycloak/keycloak:latest	8080
mysql	mysql:8	3306
zookeeper	confluentinc/cp-zookeeper	2181
kafka	confluentinc/cp-kafka	9092
prometheus	prom/prometheus	9090
grafana	grafana/grafana	3000
zipkin	openzipkin/zipkin	9411

☒ Travail Demandé

1. Établir une architecture technique du projet montrant l'ensemble des micro-services, leurs interactions synchrones (OpenFeign + Resilience4J) et asynchrones (Kafka), ainsi que le stack de monitoring
2. Créer un projet incluant : `keynote-service`, `conference-service`, `gateway-service`, `discovery-service`, `config-service`, `notification-service`
3. Développer et tester les micro-services techniques : `discovery-service`, `gateway-service`, `config-service`
4. Développer et tester `keynote-service` : Entités, Repository, Service, DTO, Mapper (MapStruct), RestController, Swagger, publication d'événements Kafka sur le topic `keynote-events`
5. Développer et tester `conference-service` : Entités (Conference, Review, Inscription), Repository, Service, DTO, Mapper, RestController, OpenFeign vers keynote-service, Circuit Breaker Resilience4J (patterns Circuit Breaker + Retry + TimeLimiter), publication d'événements Kafka, Swagger
6. Développer et tester `notification-service` : consommation des événements Kafka (tous les topics), persistance des notifications en base (entité Notification), exposition d'une API REST pour consulter l'historique et relancer les notifications échouées, documentation Swagger
7. Sécuriser l'application avec Keycloak (OAuth2 / OIDC) : validation JWT dans la Gateway et les micro-services, gestion des rôles ADMIN / MANAGER / USER
8. Mettre en place le stack de monitoring : Prometheus (scraping des métriques Actuator de tous les services), Grafana (dashboards JVM, HTTP, Kafka, circuit breakers Resilience4J, notifications), Zipkin (tracing distribué)
9. Déployer l'ensemble de l'application avec Docker et Docker Compose